Bài 1.1 – Python cốt lõi (1): biến, list, dict, vòng lặp, comprehension

> **Giai đoạn 1 – Python cho ML & Data Science** · Thời lượng: 2 buổi · Hình thức mở đầu: *Mini-game đoán kết quả*

🎯 Sau bài này bạn sẽ

- Dùng được các kiểu dữ liệu cơ bản: số, chuỗi, đúng/sai.
- Lưu trữ dữ liệu bằng **list** và **dict**, hai "cái hộp" xuất hiện trong mọi dự án ML.
- Viết vòng lặp và điều kiện để lọc dữ liệu.
- Viết **comprehension**: lọc và biến đổi dữ liệu chỉ trong 1 dòng.

Chia buổi:

- **Buổi 1:** Mục 1 → 5 (biến, list, dict).
- **Buổi 2:** Mục 6 → 10 (vòng lặp, comprehension, bài tập).

Bối cảnh xuyên suốt: bạn đang viết phần "gợi ý món ăn" cho một app giao đồ ăn.

---

1. Mini-game: đoán kết quả (5 phút)

Đọc từng đoạn code, **ghi ra giấy** kết quả bạn đoán. Sau đó mới dán vào Colab để kiểm tra. Mỗi câu đúng được 1 điểm.

```
# Câu 1
a = [1, 2, 3]
b = a
b.append(4)
print(a)

# Câu 2
print(7 / 2, 7 // 2, 7 % 2)

# Câu 3
print("3" + "4", 3 + 4)

# Câu 4
x = [n * 2 for n in range(5) if n % 2 == 0]
print(x)
```

Kết quả thật:

```
[1, 2, 3, 4]
3.5 3 1
```

```
34 7
[0, 4, 8]
```

- **Câu 1:** Ngạc nhiên chưa? `b = a` **không tạo bản sao**. `a` và `b` là hai cái tên cùng chỉ vào **một** chiếc hộp. Sửa qua `b` thì `a` cũng thấy. Muốn bản sao thật: `b = a.copy()`.
- **Câu 2:** `/` là chia thường. `//` là chia lấy phần nguyên. `%` là chia lấy số dư.
- **Câu 3:** Chuỗi cộng nhau là **nối chữ**. Số cộng nhau là **cộng toán học**.
- **Câu 4:** Bạn sẽ hiểu ở mục 7.

Được mấy điểm? Dưới 2 điểm là hoàn toàn bình thường. Bài này sinh ra để giải thích từng câu.

---

2. Vì sao học Python trước khi học ML?

Mọi dự án ML đều đi qua cùng một vòng:

```
Đọc dữ liệu → Làm sạch → Biến đổi → Đưa vào mô hình → Đọc kết quả
     |            |           |              |               |
   file        if/else     vòng lặp       list/dict        print
```

Thư viện ML lo phần "mô hình". **Mọi thứ xung quanh là Python thuần.** Nếu yếu Python, bạn sẽ kẹt ở bước làm sạch dữ liệu, chứ không phải ở thuật toán.

---

3. Biến và kiểu dữ liệu

Variable (Biến): một cái tên gắn với một giá trị, giống nhãn dán lên hộp.

```
ten_mon = "Phở bò"          # chuỗi (str): chữ, đặt trong dấu nháy
gia = 55000                 # số nguyên (int)
diem_danh_gia = 4.7         # số thực (float): có phần thập phân
con_hang = True             # đúng/sai (bool): chỉ có True hoặc False

# type() cho biết kiểu dữ liệu của một giá trị
print(type(ten_mon), type(gia), type(diem_danh_gia), type(con_hang))

# f-string: chèn biến vào câu. {gia:,} thêm dấu phẩy ngăn cách hàng nghìn
print(f"{ten_mon} giá {gia:,} đồng, đánh giá {diem_danh_gia} sao")
```

Kết quả mẫu:

```
<class 'str'> <class 'int'> <class 'float'> <class 'bool'>
Phở bò giá 55,000 đồng, đánh giá 4.7 sao
```

Liên hệ ML: cột "giá" trong dataset là số. Cột "quận" là chuỗi. Mô hình ML **chỉ hiểu số**, nên sau này ta phải biến chuỗi thành số (sẽ học kỹ ở bài 1.6).

---

4. List (Danh sách)

- **Định nghĩa:** Một dãy giá trị **có thứ tự**, đặt trong dấu `[ ]`.
- **Ví dụ đời thực:** Thực đơn quán ăn: món 1, món 2, món 3...

4.1. Đánh số vị trí (index)

Python đếm **từ 0**, không phải từ 1. Đếm ngược thì dùng số âm.

```
thuc_don =  [ "Phở bò", "Bún chả", "Cơm tấm", "Bánh mì" ]
index:           0          1          2          3
index âm:       -4         -3         -2         -1
```

```
thuc_don = ["Phở bò", "Bún chả", "Cơm tấm", "Bánh mì"]

print(thuc_don[0])      # phần tử đầu tiên
print(thuc_don[-1])     # phần tử cuối cùng
print(thuc_don[1:3])    # cắt lát (slicing): lấy từ vị trí 1 đến TRƯỚC vị trí 3

thuc_don.append("Bún bò Huế")     # thêm vào cuối danh sách
print(len(thuc_don), thuc_don)    # len() đếm số phần tử
```

Kết quả mẫu:

```
Phở bò
Bánh mì
['Bún chả', 'Cơm tấm']
5 ['Phở bò', 'Bún chả', 'Cơm tấm', 'Bánh mì', 'Bún bò Huế']
```

Mẹo nhớ slicing `[1:3]`: lấy vị trí đầu, **bỏ** vị trí cuối. Giống khoảng `[1, 3)` trong toán.

4.2. Các thao tác hay dùng

Thao tác	Code	Ý nghĩa
Thêm cuối	`ds.append(x)`	Thêm x vào cuối
Xóa theo giá trị	`ds.remove(x)`	Xóa x đầu tiên gặp
Kiểm tra có không	`x in ds`	Trả về True/False
Sắp xếp	`sorted(ds)`	Trả về list mới đã sắp xếp

Thao tác	Code	Ý nghĩa
Tổng, nhỏ nhất, lớn nhất	`sum(ds)`, `min(ds)`, `max(ds)`	Dùng cho list số

---

5. Dict (Từ điển)

- **Định nghĩa:** Tập hợp các cặp **khóa** → **giá trị** (key → value), đặt trong `{ }`.
- **Ví dụ đời thực:** Một tấm thẻ thông tin món ăn: "tên: Phở bò", "giá: 55.000", "đánh giá: 4,7".

```
mon = {"ten": "Phở bò", "gia": 55000, "danh_gia": 4.7, "cay": False}

print(mon["gia"])            # lấy giá trị theo khóa
mon["gia"] = 60000           # sửa giá trị
mon["so_luot_ban"] = 1200    # thêm khóa mới
print(mon)

# .get(): lấy giá trị, nếu không có khóa thì trả về giá trị mặc định (không báo
lỗi)
print(mon.get("calo", "chưa có dữ liệu"))
```

Kết quả mẫu:

```
55000
{'ten': 'Phở bò', 'gia': 60000, 'danh_gia': 4.7, 'cay': False, 'so_luot_ban':
1200}
chưa có dữ liệu
```

List vs Dict

	List	Dict
Lấy phần tử bằng	Vị trí (0, 1, 2...)	Tên khóa ("gia", "ten"...)
Ví dụ	Danh sách 1.000 món ăn	Thông tin chi tiết của 1 món
Trong ML giống	Một **cột** dữ liệu	Một **dòng** dữ liệu

Kết hợp hai thứ = một bảng dữ liệu. List chứa nhiều dict, mỗi dict là một dòng:

```
ds_mon = [
    {"ten": "Phở bò",    "gia": 55000,  "danh_gia": 4.7},
    {"ten": "Bún chả",   "gia": 45000,  "danh_gia": 4.5},
    {"ten": "Cơm tấm",   "gia": 40000,  "danh_gia": 4.2},
    {"ten": "Bánh mì",   "gia": 20000,  "danh_gia": 4.8},
    {"ten": "Lẩu thái",  "gia": 250000, "danh_gia": 4.0},
]
```

Tương đương bảng:

ten	gia	danh_gia
Phở bò	55000	4.7
Bún chả	45000	4.5
Cơm tấm	40000	4.2
Bánh mì	20000	4.8
Lẩu thái	250000	4.0

Đây chính là hình dạng của **DataFrame** trong Pandas (sẽ học kỹ ở bài 1.4).

Tuple và Set (biết để đọc code người khác)

```
toa_do = (10.77, 106.70)   # tuple: giống list nhưng KHÔNG sửa được, dùng ( )
print(toa_do[0])

mon_da_an = ["Phở bò", "Bánh mì", "Phở bò", "Bún chả", "Bánh mì"]
print(set(mon_da_an), len(set(mon_da_an)))  # set: tự loại bỏ phần tử trùng
```

Kết quả mẫu (thứ tự trong set có thể khác mỗi lần chạy):

```
10.77
{'Bún chả', 'Bánh mì', 'Phở bò'} 3
```

`len(set(...))` là cách nhanh nhất để đếm "có bao nhiêu giá trị khác nhau".

---

6. Điều kiện và vòng lặp

6.1. Bài toán

Khách có ngân sách 50.000đ. Gợi ý món **vừa túi tiền và được đánh giá từ 4,5 sao**.

6.2. Code

```
ngan_sach = 50000

for m in ds_mon:                                    # lần lượt lấy từng món ra, gọi
là m
    if m["gia"] <= ngan_sach and m["danh_gia"] >= 4.5:   # thỏa CẢ HAI điều kiện
        print("Gợi ý:", m["ten"])
    elif m["gia"] <= ngan_sach:                     # elif = "nếu không thì nếu"
        print("Tạm được:", m["ten"])
```

```
    else:                                          # mọi trường hợp còn lại
        print("Vượt ngân sách:", m["ten"])
```

Kết quả mẫu:

```
Vượt ngân sách: Phở bò
Gợi ý: Bún chả
Tạm được: Cơm tấm
Gợi ý: Bánh mì
Vượt ngân sách: Lẩu thái
```

Lưu ý thụt lề: Python dùng **thụt lề (4 dấu cách)** thay cho dấu `{ }` để biết dòng nào nằm trong vòng lặp. Thụt sai là lỗi `IndentationError`.

6.3. Hai mẫu vòng lặp hay gặp

```
# Mẫu 1: cần cả số thứ tự lẫn giá trị → enumerate
for i, m in enumerate(ds_mon, start=1):    # i chạy từ 1
    print(i, m["ten"])

# Mẫu 2: cộng dồn (accumulator)
tong = 0                                   # khởi tạo biến tổng = 0
for m in ds_mon:
    tong += m["gia"]                       # viết tắt của: tong = tong + m["gia"]
print("Giá trung bình:", tong / len(ds_mon))
```

Kết quả mẫu:

```
1 Phở bò
2 Bún chả
3 Cơm tấm
4 Bánh mì
5 Lẩu thái
Giá trung bình: 82000.0
```

Để ý: giá trung bình **82.000đ**, cao hơn 4/5 món! Vì món Lẩu thái 250.000đ kéo trung bình lên. Hiện tượng này gọi là **outlier (giá trị ngoại lai)** (sẽ học kỹ ở bài 1.6 và 1.7).

---

7. Comprehension: lọc và biến đổi trong 1 dòng

7.1. Trực giác

Comprehension giống câu nói tự nhiên:

```
[ m["ten"]     for m in ds_mon      if m["gia"] <= ngan_sach ]
  ─┬──────     ──────┬──────      ──────────┬──────────
  LẤY CÁI GÌ   TỪ ĐÂU               VỚI ĐIỀU KIỆN GÌ
  "tên món"    "trong danh sách"    "giá trong ngân sách"
```

7.2. So sánh hai cách viết

```python
# Cách dài: 4 dòng
goi_y = []
for m in ds_mon:
    if m["gia"] <= ngan_sach:
        goi_y.append(m["ten"])

# Cách comprehension: 1 dòng, kết quả y hệt
goi_y = [m["ten"] for m in ds_mon if m["gia"] <= ngan_sach]
print(goi_y)
```

Kết quả mẫu:

```
['Bún chả', 'Cơm tấm', 'Bánh mì']
```

7.3. Thêm ví dụ

```python
# Dict comprehension: tạo dict {tên: giá}
gia_theo_ten = {m["ten"]: m["gia"] for m in ds_mon}
print(gia_theo_ten)

# Biến đổi mọi phần tử: đổi giá sang USD (giả sử 1 USD = 25.000đ), làm tròn 2 số lẻ
gia_usd = [round(m["gia"] / 25000, 2) for m in ds_mon]
print(gia_usd)
```

Kết quả mẫu:

```
{'Phở bò': 55000, 'Bún chả': 45000, 'Cơm tấm': 40000, 'Bánh mì': 20000, 'Lẩu thái': 250000}
[2.2, 1.8, 1.6, 0.8, 10.0]
```

Giờ quay lại **câu 4 của mini-game**: `[n * 2 for n in range(5) if n % 2 == 0]`.

- `range(5)` cho các số 0, 1, 2, 3, 4.
- Giữ lại số chẵn: 0, 2, 4.

- Nhân đôi: **0, 4, 8**.

> ⚠️ Đừng lạm dụng. Nếu comprehension dài quá 1 dòng hoặc có nhiều `if` lồng nhau, hãy viết vòng lặp thường cho dễ đọc.

---

8. Ứng dụng: chuẩn bị dữ liệu cho ML

Trong ML, bạn sẽ liên tục làm những việc như:

Việc cần làm	Công cụ Python vừa học
Lọc bỏ dòng dữ liệu lỗi	`if` + comprehension
Đếm số loại khác nhau trong một cột	`len(set(...))`
Đổi đơn vị một cột (VNĐ → triệu đồng)	comprehension biến đổi
Lưu cấu hình mô hình	dict: `{"learning_rate": 0.01, "epochs": 10}`
Lưu kết quả qua từng lần thử	list: `loss_moi_vong = [0.9, 0.6, 0.4]`

Ở bài 1.3 và 1.4, NumPy và Pandas sẽ làm những việc này **nhanh hơn hàng trăm lần**. Nhưng chúng dùng đúng tư duy bạn vừa học.

---

9. Bài tập

Dùng `ds_mon` ở mục 5.

1. In ra tên món **đắt nhất**. (Gợi ý: `max(ds_mon, key=lambda m: m["gia"])`, `lambda` sẽ học kỹ ở bài 1.2.)
2. Dùng comprehension tạo list tên các món có đánh giá **trên 4,5**.
3. Tính giá trung bình của các món **không tính Lẩu thái**. So sánh với 82.000đ.
4. Tạo dict `{tên món: "Rẻ" hoặc "Đắt"}`. Món dưới 50.000đ là "Rẻ".
5. Cho `don_hang = ["Phở bò", "Bánh mì", "Phở bò", "Phở bò", "Bún chả"]`. Đếm mỗi món được gọi bao nhiêu lần, kết quả dạng dict. (Gợi ý: dùng vòng lặp và `dem.get(ten, 0) + 1`.)

Đáp án

> Mẹo Notion: chọn tiêu đề "Đáp án" và bấm `Turn into → Toggle heading 3` để giấu đi.

```
# 1
print(max(ds_mon, key=lambda m: m["gia"])["ten"])          # Lẩu thái

# 2
print([m["ten"] for m in ds_mon if m["danh_gia"] > 4.5])  # ['Phở bò', 'Bánh mì']

# 3
gia = [m["gia"] for m in ds_mon if m["ten"] != "Lẩu thái"]
print(sum(gia) / len(gia))                                 # 40000.0 → rẻ hơn
82.000đ rất nhiều
```

```python
# 4
print({m["ten"]: ("Rẻ" if m["gia"] < 50000 else "Đắt") for m in ds_mon})
# {'Phở bò': 'Đắt', 'Bún chả': 'Rẻ', 'Cơm tấm': 'Rẻ', 'Bánh mì': 'Rẻ', 'Lẩu thái':
'Đắt'}

# 5
don_hang = ["Phở bò", "Bánh mì", "Phở bò", "Phở bò", "Bún chả"]
dem = {}
for ten in don_hang:
    dem[ten] = dem.get(ten, 0) + 1     # chưa có thì coi như 0, rồi cộng 1
print(dem)                              # {'Phở bò': 3, 'Bánh mì': 1, 'Bún chả':
1}
```

---

10. Flashcard

Mặt trước	Mặt sau
Python đếm index bắt đầu từ?	0. Phần tử cuối là `[-1]`.
`ds[1:3]` lấy những vị trí nào?	Vị trí 1 và 2 (bỏ vị trí 3).
`b = a` với list có tạo bản sao không?	Không. Cả hai cùng chỉ một list. Dùng `a.copy()`.
List vs Dict?	List lấy theo vị trí. Dict lấy theo tên khóa.
Lấy giá trị dict an toàn khi có thể thiếu khóa?	`d.get("khoa", mac_dinh)`
Cấu trúc comprehension?	`[LẤY_GÌ for x in TỪ_ĐÂU if ĐIỀU_KIỆN]`
`7 // 2` và `7 % 2`?	3 (phần nguyên) và 1 (số dư).
Đếm số giá trị khác nhau?	`len(set(ds))`

Bài tiếp theo: 1.2 – Hàm, class, đọc/ghi file.